

Artículo: Solving no-wait job-shop scheduling problems using a multi-start simulated annealing with bi-directional shift timetabling algorithm

Oscar Andres Diaz Gonzalez
oadiazg@eafit.edu.co, Universidad EAFIT

6 de mayo de 2026

Descripción del Problema

El problema descrito en el artículo Ying & Lin (2020) seleccionado es el mismo problema que trabajamos en clase (NWJSSP) con la única diferencia en la función objetivo tratada, pues en el artículo se trabaja con la función objetivo de minimizar el tiempo de finalización de la última operación del último trabajo completado.

Problema NWJSSP: (Tomado del Anexo 1 del Trabajo 1 de Heurística)

El problema de programación job shop sin esperas implica un conjunto de máquinas M y un conjunto de trabajos J que deben ser procesados en estas máquinas. Cada trabajo $j \in J$ se caracteriza por una fecha de liberación r_j y un conjunto de operaciones O . Cada operación $u \in O$ también se caracteriza por la máquina en la que se procesa (m_u), así como por el tiempo de procesamiento que requiere (p_{ju}).

Se deben cumplir las siguientes restricciones:

1. Cada trabajo tiene su propia secuencia de operaciones, la cual indica su propia ruta de procesamiento; es decir, la operación u solo puede ser procesada después de la operación $u - 1$ ($u \in O, u > 1$).
2. Ninguna operación puede ser interrumpida una vez que ha comenzado.
3. Ningún trabajo puede ser procesado en más de una máquina simultáneamente.
4. Ninguna máquina puede procesar más de un trabajo simultáneamente.
5. Ningún trabajo puede volver a entrar en máquinas previamente visitadas.
6. No se permite tiempo de espera entre dos operaciones consecutivas del mismo trabajo.

El objetivo es encontrar un programa factible que **minimice el tiempo de finalización del último trabajo (adaptado al problema del artículo seleccionado)**.

Algoritmo Propuesto

El algoritmo propuesto es un *Multi-Start Recocido Simulado* (MSA) que incorpora un procedimiento de *Bidirectional Shift Timetabling* y un mecanismo simple de retraso de una unidad de tiempo para resolver conflictos de cuello de botella, los cuales definen la programación de los tiempos de inicio de la secuencia de trabajos, es decir, definen el valor de la función objetivo de la secuencia.

Los parámetros del algoritmo son los siguientes:

-
- p : Tamaño de la población inicial de soluciones.
 - T_0 : Temperatura inicial del recocido simulado.
 - I_{ter} : Número de iteraciones realizadas a cada temperatura.
 - α : Factor de enfriamiento de la temperatura.
 - $N_{non-improving}$: Número máximo de iteraciones del Recocido simulado sin mejora de la mejor solución encontrada.

A continuación, se describe el funcionamiento general del algoritmo:

Paso 1:

Se inicializa la temperatura $T = T_0$ y dos contadores $R = 0$ y $N = 0$. El contador R controla el número de enfriamientos sin mejora de la mejor solución, mientras que N cuenta las iteraciones realizadas a la temperatura actual.

Paso 2:

Se genera una población inicial $P = \{\pi_1, \pi_2, \dots, \pi_p\}$ de tamaño p . Cada solución π_q se representa mediante una lista de $n + 2$ elementos, donde n es el número de trabajos. (ejemplo de una solución con 7 trabajos):

$$\pi_q = [7, 4, 5, 6, 1, 3, 2 \mid 0 \mid 3]$$

- Las primeras n posiciones contienen una permutación aleatoria de los n trabajos (generada uniformemente).
- La posición $n + 1$ indica la dirección del timetabling: 0 para *Forward* y 1 para *Backward*. (valor elegido de manera aleatoria con probabilidad uniforme)
- La posición $n + 2$ indica el trabajo cuya primera operación será retrasada una unidad de tiempo (valor generado aleatoriamente entre 1 y $4n$, lo que da un 25 % de probabilidad de retrasar cada trabajo).

Se identifica la mejor solución de la población inicial π_{best} y se guarda su valor objetivo $F_{best} = z(\pi_{best})$.

Paso 3:

Se ejecutan I_{ter} iteraciones del recocido simulado de la siguiente forma:

- Se selecciona aleatoriamente una solución inicial $\pi_q \in P$.
- Se perturba la solución inicial generando una solución vecina π'_q aplicando uno de los siguientes operadores con igual probabilidad: *Swap*, *Insertion*, *2-Swap*, *2-Insertion* o *Inversion*. Además, con probabilidad del 1 % se cambia la dirección del timetabling y con otro 1 % se modifica el trabajo a retrasar.
- Se calcula $\Delta_q = z(\pi'_q) - z(\pi_q)$. Si $\Delta_q < 0$, se acepta π'_q . Si $\Delta_q \geq 0$, se acepta con probabilidad $\exp(-\Delta_q/T)$.
- Si $z(\pi_q) < F_{best}$, se aplica una búsqueda local tipo *best improvement* (con movimientos *Swap*) sobre π_q , se actualiza π_{best} y F_{best} , se reinicia el contador R y se copia la mejor solución a toda la población.

Después de I_{ter} iteraciones se enfría la temperatura ($T \leftarrow \alpha T$), se incrementa R y se reinicia N . El algoritmo termina cuando se alcanzan $N_{non-improving}$ enfriamientos sin mejorar la mejor solución encontrada.

Los componentes claves del algoritmo es el procedimiento **Bidirectional Shift Timetabling** y la posibilidad de retrasar una unidad de tiempo la primera operación de un trabajo seleccionado, donde el primero se encarga de asignar los tiempos de inicio de las operaciones respetando la restricción *no-wait*. Este procedimiento permite dos formas de decodificación:

- **Forward Timetabling:** Se programan los trabajos en el orden de determinado en la secuencia generada buscando programar la primera actividad en el menor tiempo posible cumpliendo la restricción No-wait. (De la misma manera en que normalmente se ha intentado resolver el problema)
- **Backward Timetabling:** Se programan los trabajos en el orden inverso a la secuencia generada aleatoria, además se busca programar lo más tarde posible, es decir, que la última actividad sea completada lo más tarde posible cumpliendo la restricción No-wait.

Además, el retrasar una unidad de tiempo la primera operación de un trabajo seleccionado, la cual permite resolver cuellos de botella que aparecen en las programaciones sin retraso, ampliando significativamente el espacio de soluciones factibles y permitiendo encontrar mejores soluciones para una misma secuencia de trabajos.

Es decir, estos dos componentes claves son la forma en que se programan los trabajos de cada solución para posteriormente calcular el valor z correspondiente a la secuencia.

La combinación del esquema multi-start de recocido simulado con el bidirectional shift timetabling y el retraso es lo que otorga al algoritmo propuesto Ying & Lin (2020) su capacidad para explorar de manera efectiva el espacio de soluciones del problema *NWJSSP*.

Referencias

Ying, Kuo-Ching, & Lin, Shih-Wei. 2020. Solving no-wait job-shop scheduling problems using a multi-start simulated annealing with bi-directional shift timetabling algorithm. *Computers & Industrial Engineering*, **146**, 106615.